

DiLLS: Interactive Diagnosis of LLM-based Multi-agent Systems via Layered Summary of Agent Behaviors

Rui Sheng*

Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
Hong Kong, China
rshengac@connect.ust.hk

Yukun Yang*

School of Software Engineering
Tongji University
Shanghai, China
yangyyk@tongji.edu.cn

Chuhan Shi

Southeast University
Nanjing, China
cshiag@connect.ust.hk

Yanna Lin

Cheriton School of Computer Science
University of Waterloo
Waterloo, Canada
yanna.lin@uwaterloo.ca

Zixin Chen

Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
Hong Kong, China
zchendf@connect.ust.hk

Huamin Qu

Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
Hong Kong, China
huamin@cse.ust.hk

Furui Cheng[†]

Department of Computer Science
ETH Zürich
Zürich, Switzerland
furui.cheng@inf.ethz.ch

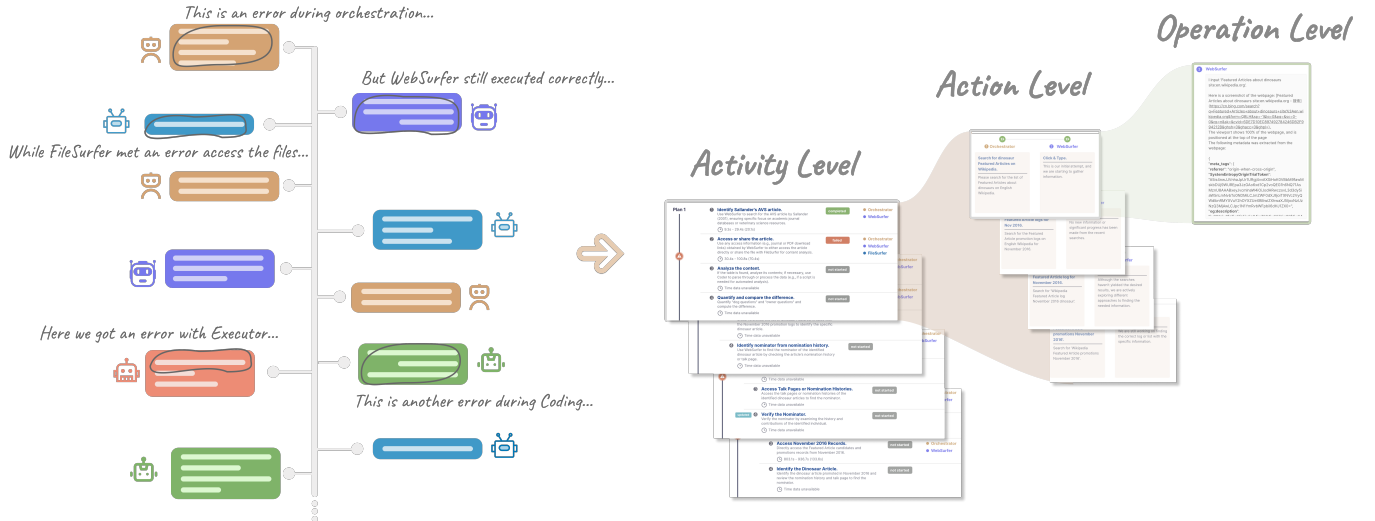


Figure 1: We designed a layered exploration approach and implemented a system, *DiLLS*, to assist AI developers in exploring agent behaviors across various layers for multi-agent system diagnosis.

*Equal contribution

[†]Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License.
CHI '26, Barcelona, Spain

Abstract

Large language model (LLM)-based multi-agent systems have demonstrated impressive capabilities in handling complex tasks. However, the complexity of agentic behaviors makes these systems difficult to

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2278-3/26/04
<https://doi.org/10.1145/3772318.3790815>

understand. When failures occur, developers often struggle to identify root causes and to determine actionable paths for improvement. Traditional methods that rely on inspecting raw log records are inefficient, given both the large volume and complexity of data. To address this challenge, we propose a framework and an interactive system, *DiLLS*, designed to reveal and structure the behaviors of multi-agent systems. The key idea is to organize information across three levels of query completion: activities, actions, and operations. By probing the multi-agent system through natural language, *DiLLS* derives and organizes information about planning and execution into a structured, multi-layered summary. Through a user study, we show that *DiLLS* significantly improves developers' effectiveness and efficiency in identifying, diagnosing, and understanding failures in LLM-based multi-agent systems.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; • **Computing methodologies** → **Multi-agent systems**.

Keywords

Large Language Models, Multi-agent Systems, Interactive Diagnosis

ACM Reference Format:

Rui Sheng, Yukun Yang, Chuhan Shi, Yanna Lin, Zixin Chen, Huamin Qu, and Furui Cheng. 2026. *DiLLS: Interactive Diagnosis of LLM-based Multi-agent Systems via Layered Summary of Agent Behaviors*. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3772318.3790815>

1 Introduction

There is a growing interest in organizing large language models (LLMs) into agentic architectures to handle complex tasks [34, 55]. Such multi-agent systems harness the capabilities of LLMs to interpret human instructions, plan execution strategies, and delegate responsibilities across individual agents. When further augmented with features such as short-term or long-term memory and external tool use, these agents can collaborate more effectively to solve complex problems. Promising applications have already emerged across domains including web searching [1, 9], scientific research [10–12], and software development [37, 44].

While LLM-based multi-agent systems introduce promising capabilities in autonomous task solving, monitoring and diagnosing their behaviors remain challenging [7]. Unlike traditional autonomous systems with human-authored running logic and well-defined execution paths, LLM-based agentic systems operate with a high degree of flexibility in task scheduling and execution, making their behaviors hard to track, interpret, and regularize. The involvement of multiple agents further amplifies this complexity, introducing additional challenges in coordinating and understanding their interactions. When failures occur, it is often unclear how to identify, understand, or resolve the underlying issues.

Conventional debugging relies on scanning agent logs and execution histories, but in LLM-based systems, interactions are typically conducted through natural language dialogues between agents. As a result, the logs are highly unstructured and verbose, frequently spanning hundreds of exchanges. Reading through such transcripts

is not only time-consuming but also makes it difficult to discern agent intentions, coordination strategies, or failure points. Additionally, the diversity of agent behaviors, ranging from web searching, coding, to file reading, can overwhelm developers with excessive details and obscure the context of the whole problem-solving process. This further complicates the task of diagnosing and understanding system failures. Although agent management platforms such as AutoGen Studio [6], CrewAI [5], Voiceflow [53], LangGraph [26], and LangSmith [38] provide visibility into agent execution histories, deriving root causes of failures and generating hypotheses about how to improve systems remains an open challenge.

To better understand the practical challenges of diagnosing LLM-based multi-agent systems, we conduct a formative study with eight experienced developers. Our findings reveal that those experts strongly demand high-level summaries of agents' planning and execution behaviors to gain an overview of potential failure causes. Moreover, an efficient interactive visualization interface is needed to help developers locate erroneous execution histories and interpret system failures more efficiently.

Building on insights from our formative study, we propose a framework to derive, organize, and present agentic behaviors within a centralized architecture. The major challenge lies in determining the appropriate layers of abstraction that provide a coherent and structured organization of behaviors. We were inspired by Activity Theory (AT), a framework for understanding human behaviors and practices, where an activity is decomposed into three levels: the overarching activity, the actions as goal-directed processes supporting the activity, and operations as the concrete executions. Inspired by this hierarchy, we employ natural language probing techniques to analyze multi-agent system behaviors across three layers: overall planning and execution, action arrangement and outcomes, and logs for concrete operations. Based on this layered representation, we developed a tool, *DiLLS*, to facilitate the exploration and interpretation of agent behaviors across these three levels.

We evaluate *DiLLS* through a user study with 12 multi-agent system developers. Participants were asked to diagnose an erroneous multi-agent system using both *DiLLS* and a baseline system simplified from *DiLLS* that only presents basic operation-level logs, report their findings, and provide both qualitative and quantitative feedback. Results show that our system achieves significantly higher efficiency in error identification, as evidenced by the greater number of potential model errors detected by participants. Moreover, *DiLLS* leads to lower mental demand and workload, demonstrating its effectiveness in supporting the diagnosis of complex multi-agent behaviors. Our contributions include:

- We conduct a formative study with eight AI developers to uncover how practitioners currently diagnose failures in multi-agent systems and the challenges they face.
- We propose a framework and a system, *DiLLS*, that structures behavioral information in multi-agent systems to support developers in understanding and diagnosing case-level failures.
- We validate the effectiveness of *DiLLS* through a user study, showing that it enables practitioners to identify failures more confidently and comprehensively while reducing their stress.

2 Background and Related Work

In this section, we first introduce the background of LLM-based multi-agent systems, including their working mechanisms and design components. Specifically, we focus on the four types of interaction architectures among agents. After that, we elaborate on the current diagnostic tools for agentic systems, clarifying their limitations as well as the innovations and scope of our tool.

2.1 LLM-Based Multi-Agent Systems

Advances in large language models (LLMs) have catalyzed the development of multi-agent systems capable of tackling complex tasks across diverse domains, such as web search [40], scientific research [49], software development [8, 14, 21], and societal and classroom simulations [17, 35, 42, 45, 54]. These systems typically comprise multiple specialized agents, each with distinct roles, prompts, or tools. They collaborate to decompose tasks, delegate subtasks, and leverage external resources or memory [16, 32, 42, 60]. For example, Magentic-One, a multi-agent system developed by Microsoft, features an Orchestrator that directs specialized agents like WebSurfer, FileSurfer, and Coder to accomplish intricate tasks [9]. A practical application could be automating research: the WebSurfer gathers web-based information, the Coder analyzes the data, and the FileSurfer compiles the findings into a structured report. By distributing workloads and combining complementary capabilities, multi-agent approaches often outperform single-agent counterparts in addressing complex use queries [3, 30, 47, 60].

To streamline the construction and orchestration of these LLM-based multi-agent systems, multiple frameworks have been proposed [5, 26, 31, 57, 58]. One of the widely accepted agent design frameworks was introduced by Li et al. [32], which defines five key components: the creation of various agents (profile), the ability to perceive the environment (perception) and communicate with other agents (mutual interaction), the ability to respond to and complete complex tasks (self-action), and the ability to enhance their intelligence through self-reflection (evolution).

Additionally, Li et al. [32] also summarize four types of architectures for agent interactions: hierarchical, decentralized, centralized, and shared message pool (Fig. 2). First, hierarchical architecture arranges agents in a tree-like structure, allowing higher-level agents to delegate tasks to lower-level counterparts. For example, MDAgents [24] introduces an approach where a Moderator managed multi-disciplinary clinicians. Depending on the different medical diagnosis requirements, the Moderator can recruit different clinicians, thereby improving the precision of disease treatment. Next, decentralized architecture enables agents to operate autonomously and communicate directly. For instance, Chen et al. [4] propose a framework called DMAS, where every robot’s LLM agent can engage in task planning through turn-taking dialogues. This approach enables agents to express their views independently while integrating feedback from others, thereby enhancing collaboration for task completion. Then, centralized architecture relies on a single authority to oversee and manage all agents. Magentic-one [9] utilizes this type of communication architecture, where web agents, file agents, coders, and other agent workers all respond to the orchestration and direction of a central orchestrator. Finally, shared

message pool architecture allows agents to interact through a common set of messages, promoting efficient information exchange. An illustrative example is MetaGPT [15], which features a shared message pool that enables agents to continuously access and gather pertinent information, thus improving their collaborative efforts and overall communication efficiency. In our work, we focus on studying the diagnosis of centralized multi-agent systems.

2.2 Agentic System Diagnosis Tools

Diagnosis of a system refers to identifying the location and cause of failures within that system [18]. The insights gained from diagnosis can serve to improve the system in subsequent stages. While a range of diagnosis tools exist for LLM-based single-agent systems, such as covering prompt-testing interfaces [7, 20, 50] and output-comparison systems [22, 23, 46], they often fail to address the broader challenges of multi-agent collaboration, which demand synchronized, system-wide analysis of multiple agents, their outputs, and their inter-communication [51].

Currently, platforms like AutoGen Studio [6], CrewAI [5], Voiceflow [53], and LangGraph [26], use chat-style interfaces to help developers interact with AI agents. These tools are capable of streaming the behaviors and generated messages (such as code snippets and created files) from the multi-agent system into the interface. Furthermore, several of them, like AutoGen Studio, also offer an overview interface that allows users to monitor metrics such as the number of times different agents are invoked and the amount of tokens consumed. However, the overview provided by these tools often falls short in helping users identify the key points that lead to task failures. Instead, the focus tends to be on resource consumption issues, such as excessive execution time. Additionally, this streamlined presentation of information can overwhelm developers, making it difficult to effectively understand the overall logic of task execution, thereby hindering the diagnosis of issues. Other works, such as LangSmith [38], have introduced visualization methods for presenting the multi-agent execution process, which can enhance developers’ ability to locate and understand failures. However, it primarily focuses on improving the visual appeal of the original execution sequence. From those sequentially arranged execution details, developers still face challenges in effectively achieving a high-level understanding of the task execution, such as identifying frequent failure points and recognizing where multiple attempts were made. Recently, AGDebugger [7] has been proposed to assist developers in debugging LLM-based multi-agent systems; however, it mainly helps AI developers improve systems after failures have been identified and understood.

In response, there is currently a lack of effective tools to help users identify and understand failures in multi-agent behaviors. Therefore, we propose a three-layered approach based on Activity Theory [25] to organize agent self-actions and interactions, focusing on the diagnosis of centralized multi-agent systems. Our system allows developers to switch between high-level behavior summaries and deeper log inspections as needed. By mapping how plans evolve, how agents coordinate tasks, and how errors compound over multiple rounds, developers can more efficiently understand failures.

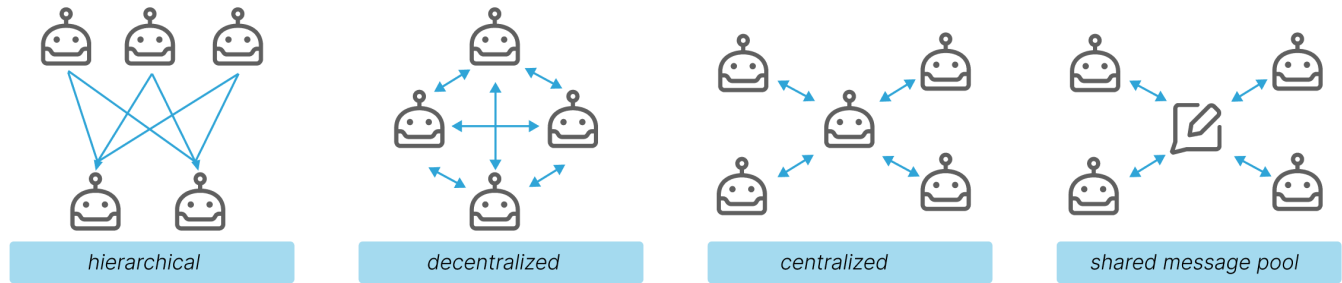


Figure 2: The four interaction architectures.

3 Formative Study

To identify pain points in locating and understanding failures for diagnosing multi-agent systems and to derive design requirements of our system, we conducted a qualitative formative study.

3.1 Participants and Procedure

We interviewed eight multi-agent project practitioners (E1–E8; 3 female, 5 male) recruited through social media and word-of-mouth referrals. The participants represented diverse roles and experience levels, including a startup founder (E1) who had led five multi-agent projects, two product developers (E2–E3), and five Ph.D. students specializing in AI or HCI (E4–E8).

During the interview, participants were asked to describe their most recent experience developing a multi-agent system. We prompted them to provide concrete examples of failures they encountered during development, along with the strategies they used to diagnose these issues. We then followed up with questions about the challenges they faced throughout the diagnostic process, encouraging them to elaborate with detailed examples. The full list of interview questions is provided in Appendix A. All interviews were audio-recorded. Each interview lasted approximately 90 minutes, and participants received US \$15 in compensation. This study was approved by the Institutional Review Board.

We analyzed the interview data using the thematic analysis proposed by Braun and Clarke [2]. All videos were transcribed, and then they were independently coded inductively by two researchers, focusing on participants’ diagnostic workflows and the challenges they encountered. During open coding, the researchers code the participants’ self-reported diagnostic activities with concrete, process-oriented codes (e.g., *forming an initial overview*, *reviewing agent logs*, and *searching for anomalous behaviors*), with each code simultaneously annotated with the associated challenges. After independent coding, the two researchers collaboratively discussed the codes to derive higher-level themes by clustering related codes. As the first goal of this analysis was to identify common steps in the diagnosis process, codes were grouped into themes primarily based on similarities in diagnostic intent and temporal position within the diagnosis workflow. When coding disagreements arose, the researchers revisited the original transcripts and engaged in iterative discussion to reconcile differing interpretations and reach consensus. This process resulted in three major diagnostic steps. The two researchers also reviewed and named these three steps. Within each diagnostic step, the two researchers then applied a similar iterative

clustering process to identify recurring challenges. These themes form the basis of the findings reported in the following section, supported by representative participant quotes.

3.2 Diagnostic Process and Challenges

Through our qualitative analysis, we identified three common steps in multi-agent system diagnosis, along with the key challenges experts encounter during this process.

Gaining an Overall Understanding. A typical initial step of multi-agent system diagnosis involves making sense of the large volumes of diverse agent behaviors to gain an overall understanding of how they address the user query. This overview can help users identify any inconsistencies in the overall task execution logic. For instance, E3 discussed the experience with a stock analysis agentic system: *“I needed to ensure that the system first gathered real-time news to assess market sentiment, then pulled historical prices to identify trends. Finally, the agentic system combined these elements to perform the analysis. Only if this workflow is correct do the specific behaviors at each step make sense.”* However, all of our experts reported a high cognitive load during this understanding process. They mentioned that they often need to carefully review information generated by agent operations (E1–E8), and take notes and reorganize the information (E2, E3, and E6) to make sense of the data, such as constructing their plan structures (E2, E6). **This challenge intensified when different contexts required different types of summaries (E1, E2, E3, E6, and E8).** For example, E6 mentioned that she sometimes needed to quickly review updates across all plans, while at other times she needed an overview of the operations within a specific plan. **Additionally, in some cases, our experts (E2, E4, and E6) even extract information not explicitly present in the logs** to enhance their understanding. For example, E2 often manually infers whether the planned actions and actual executions are consistent, since this information is not explicitly recorded in the log data. This bottom-up sense-making process was thought of as time-consuming and mentally demanding.

Locating Failures. Then, developers will search for specific agent behaviors that reveal the root causes of failures. However, experts have highlighted the challenges in determining where to focus attention first (E2, E3, E4, E6, and E7) because **this often requires examining multiple interrelated agent behaviors buried within large volumes of logs**. As E4 noted, *“For example, if the behaviors of these agents are all aimed at completing the CT*

report analysis, it is important to consider them collectively. Analyzing a single agent's behavior in isolation may not provide meaningful insights; for instance, while one action may have failed, subsequent actions could potentially compensate for that failure." In some cases, two agent behaviors may occur far apart in the log data, such as one appearing at the beginning and the other in the middle, yet still be very similar, and developers hope to analyze them together. Unfortunately, the current method requires users to manually associate the execution step purpose with specific agent operations, which is quite arduous.

Interpreting Failures with Context. Identifying one failure does not mark the end of the process. AI developers often need to interpret these failures within the broader conversational context to understand how and why the breakdown occurred. However, they often **struggle to relate failures to the surrounding context within unstructured log data** to determine the best approach for fixing those failures (E1, E3, E4, E7, and E8). For example, E4 and E7 mentioned that they may discover failures at different levels, such as some behaviors that were not supposed to execute at all since they could not achieve the task's objectives, while others were intended to execute but resulted in execution errors. E7 stated, "Some failures may have higher priority, but I ultimately need to manually categorize them. When faced with a large number of identified failures, this process is often very time-consuming."

3.3 Design Considerations

Based on our findings, we have summarized four design considerations for diagnosing multi-agent systems.

DC1 Provide multi-layer behavior summary. We found that developers traditionally rely on large, unstructured log data to obtain an overview of multi-agent execution. This creates substantial cognitive load and highlights the need for behavior summaries. Moreover, several participants reported requiring different levels of summaries depending on the context. Therefore, the system should provide multi-layer summaries of agent behaviors, enabling users to flexibly select the appropriate overview on demand. Such a multi-layer structure can also help developers integrate contextual information across different levels to better interpret identified failures.

DC2 Elicit hidden thoughts of agent behaviors. Our experts noted that when constructing an overview of the system's execution, they often needed to extract information that was not explicitly provided in the logs or model outputs. However, performing these inferences across large numbers of behaviors is highly time-consuming and prone to error. Therefore, the system should automatically surface such implicit information. For instance, we can show behaviors that concretely contributed to each step, when and why a step was revised after a failure, and how each behavior aligns with the intended workflow. This reduces users' cognitive burden and enables more precise diagnosis of coordination or reasoning issues.

DC3 Support grouping and joint analysis of different agent behaviors. Experts emphasized that diagnosing failures often requires reasoning about sets of agent behaviors that are logically or functionally related, such as those contributing to the

same subtask, pursuing the same intermediate goal, or compensating for earlier failures. However, these relationships can be implicit and scattered across large volumes of logs, forcing developers to manually infer and assemble relevant behaviors, which is cognitively demanding and error-prone. Therefore, our system should explicitly link such related agent behaviors, enabling developers to analyze them jointly.

DC4 Present the information generated by agents in a structured form. Multi-agent conversational logs often contain heterogeneous information, such as their objectives, processes, outcomes, and updates. To facilitate comprehension and navigation, our system should present these diverse types of information in a clear and structured format, such as categorized tables or hierarchical lists. Therefore, users can more efficiently navigate, filter, and relate information. This also helps them form a clear overview of the agent execution process and identify failures more effectively.

4 Framework

To address the challenges in diagnosing multi-agent systems, we propose a framework for extracting, organizing, and presenting agent behaviors at multiple levels. The framework is grounded in the progress of task completion, a design choice motivated by the diagnostic context: users care most about understanding why tasks fail and how to fix them. This section introduces the framework designs and technically explains how the required information is extracted by probing a centralized multi-agent system.

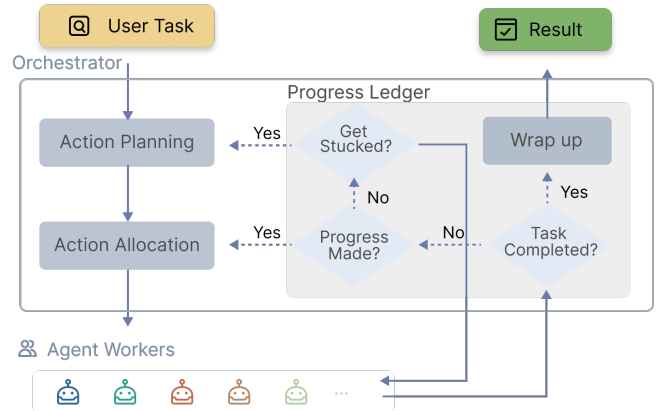


Figure 3: The execution process in centralized LLM-based multi-agent systems.

4.1 Layered Summary of Agent Behaviors

From our formative study, we found that relying solely on plan log information makes it difficult to gain a holistic understanding of agent behaviors. To address this, we aim to develop a multi-layered summary that provides a structured yet comprehensive overview of the agents' behaviors (DC1).

The key challenge in designing this summary lies in determining the layers of behaviors that carry concrete semantics and can highlight specific types of

agentic issues. We were inspired by Activity Theory (AT), a framework for understanding human behaviors and practices within socially situated contexts [27, 28]. AT has been widely adopted in human–computer interaction research to analyze the interplay between humans and technology [25].

We adapt the framing of activities in AT to describe and organize agent behaviors as a way to complement user queries. Specifically, AT conceptualizes an activity as a purposeful, socially mediated system that can be understood at multiple levels: the **activity** itself, which is driven by a broader motive; **actions**, which are goal-directed processes that realize the activity; and **operations**, which are routinized or automated procedures conditioned by the immediate context. For example, organizing a community volunteer event can be seen as an activity: the goal is to help a local community, the actions include coordinating volunteers and arranging tasks, and the operations are the fine-grained steps, such as sending emails, booking venues, and managing schedules. This hierarchical framing provides a principled lens for structuring and interpreting layers of agent behaviors, enabling us to map system-level intentions, task-level strategies, and low-level operations in ways that can reveal different types of agentic issues. This can help developers better make sense of agent behaviors and diagnose the systems.

At the activity level, we aim to provide users with a high-level understanding of the system’s approach to completing the activity, namely, *how it complements the user’s query, executes the plan, and adapts as the plan evolves*. The activity-level summary enables users to assess the overall organization of how the query is being processed and the progress made toward completing the activity. If the organization appears unsatisfactory, users can infer that the system is not capable of handling the query and thus requires substantial changes. Conversely, if the high-level organization is adequate, users can then identify specific actions where the system repeatedly fails and drill down further to locate the causes of the issues.

At the action level, we aim to support users to delve deeper into the execution of a specific action by providing an overview of all the operations performed to complete this action, including the specific operations undertaken, their success or failure, and the results returned. This enables users to access the worker agent’s operation decomposition capability and drill down to locate the erroneous operations.

At the operation level, we enable users to examine the concrete details of the system’s behavior, including the log history, agent tool usage, and web access processes. By checking the details, users could confirm their findings about the issues.

4.2 Probing Centralized Multi-agent Systems

In centralized multi-agent systems, a central agent, known as the orchestrator, manages overall task progress (Fig. 3). By maintaining a global view of the system, the orchestrator can optimize resource allocation, prevent conflicts, and ensure consistent outcomes, which makes centralized control a widely studied paradigm for critical tasks [9, 19, 41, 43, 48]. The orchestrator typically decomposes a complex task into a sequence of steps, with each step referred to as an action (action planning). It tries to complete these actions one by one by invoking specialized agents (action allocation). For example,

completing a “document reading” action might involve first calling a web-searching agent to retrieve the literature, followed by a “file-reading” agent to parse and analyze its contents. The ordered sequence of such agent operations collectively fulfills the action, contributing to the overall progress of the task. At the same time, the orchestrator needs to manage the overall progress of the task. For example, it determines the next steps by checking whether the task is completed, whether there has been no progress, or if it has entered a loop. Based on this assessment, the orchestrator decides whether to directly output results, update the planned actions itself, or continue invoking other agents.

In an activity-level summary, we aim to provide users with a high-level understanding of how the multi-agent system completes an entire activity. To achieve this, we present the plan as a unit, which includes information about plan failures, plan updates, and action progress overview in each plan. This summary helps developers quickly grasp how the multi-agent system executed the activity, enabling them to identify necessary actions that are significant and worth exploring further. The specific information extracted from agent behaviors includes the following:

- **Plan failures and updating:** We refer to a sequence of actions capable of completing an entire activity as a plan. In an activity, a multi-agent system often attempts various actions, which may occasionally lead to failures. This necessitates consideration of how to update the plan. At this layer, we provide information on instances of plan failure and their causes, as well as details on how to update the plan accordingly. This can help developers understand how the entire activity is performed more effectively.
- **Actions in each plan with their status:** The final status (completed, failed, or not started) of each action within a plan is crucial for developers to understand where the process may be stuck. This information helps users narrow down which specific action to examine closely, facilitating more efficient diagnosis.
- **Agents for each action:** Displaying which agents execute those actions provides additional context and information. This can help developers understand how an action is executed based on their expertise, without needing to delve into the internal workings of the action itself. For example, if an action involves reading a paper using a web agent, developers can intuitively understand that this action is completed through an online process. Conversely, if a file agent is used, it indicates that the reading is done locally. In certain cases, this distinction provides clarity, allowing developers to make an informed judgment about whether this action warrants further investigation. This enhances their ability to prioritize their efforts effectively.
- **Action progress:** Multiple agents may interleave their contributions to an action, making it difficult for developers to quickly understand each agent’s participation, their coordination patterns, and the resulting outcomes. This makes it challenging to pinpoint the root of failures. To address this, we provide summaries that make these aspects explicit and easier to analyze. These summaries reveal how the system navigated the action, including the successes and failures that occurred.

In an action-level summary, we enable users to delve deeper into the execution of a specific action after they identify which actions require further investigation. At this level, we provide an overview

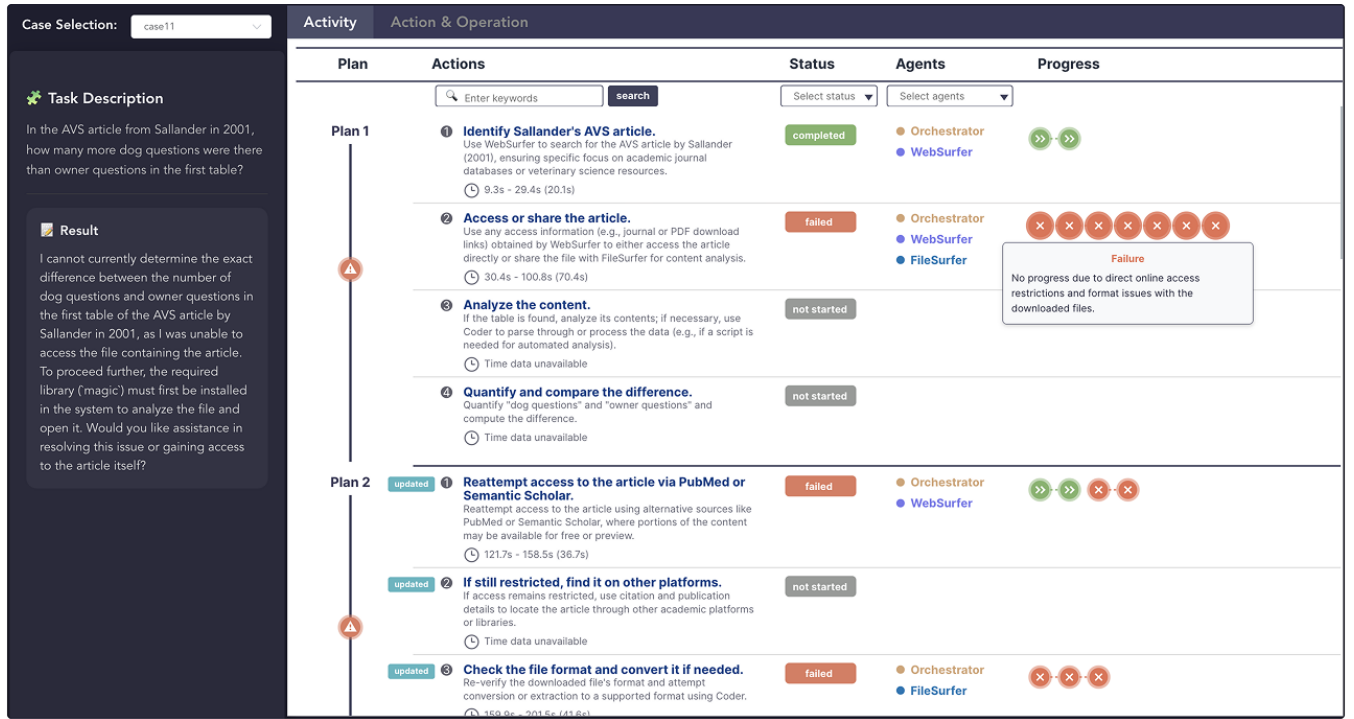


Figure 4: The Activity View primarily helps AI developers gain a high-level understanding of the plans proposed by multi-agent systems and their execution status. Based on the user feedback of our formative study, we present five types of information at this level.

of all the operations performed to complete this action, including the specific operations undertaken, their success or failure, and the results returned. The detailed information includes the following:

- **Operation description:** An overview of each operation can illuminate the workflow involved in executing the action. This enables developers to quickly grasp how the action was executed.
- **Operation type:** Providing a higher-level summary of operation descriptions can help developers effectively capture the relationships between operations in an action, aiding in identifying errors. For example, a web agent might have the capabilities to navigate URLs or read pages. If a developer discovers an issue with the “read pages” action, they can quickly validate the problem by examining the same or other similar types of actions.
- **Operation result summarization:** The concise representation of the outputs can reveal critical insights into the overall effectiveness of each operation. By summarizing the results, developers can quickly assess whether the operations achieved the desired outcomes and how they contribute to the action’s success or failure. This can help developers select which operations to drill down into for further investigation.

In operation-level details, we present the comprehensive log data for each operation, allowing developers to trace the original output of multi-agent systems.

- **Log data:** The concise representation of the outputs resulting from the operations performed by an agent.

To elicit agents’ hidden thoughts for diagnostic purposes, such as plan formulation and revision, action status, and action progress, we employ natural language probing, using structured prompts that encourage self-evaluation and progress reporting (DC2). This probing enables us to capture key information across multiple levels of the system. In addition, we perform post-hoc processing to summarize operation goals and descriptions. These prompts are adapted from the Magentic-One framework [9], a centralized multi-agent framework developed by Microsoft for solving complex tasks, which also extracts several types of information we need. The complete set of prompts is provided in the supplementary materials, where we have highlighted the modifications we made.

5 DiLLS

We introduce *DiLLS*, an interactive system that organizes task planning and execution information to support failure diagnosis. A central design consideration is providing a multi-layered summary of agent behaviors (DC1). As discussed in Section 4.1, agent behaviors are organized into three layers (i.e., activity, action, and operation) adapted from Activity Theory. *DiLLS* exposes these layers along the progression of task completion, enabling users to assess the overall activity, identify failed or stalled actions, and inspect operation-level details to verify root causes. Specifically, the Activity View offers a high-level overview of activity progress and outcomes, the Action View reveals action execution histories, and the Operation View displays raw operation-level logs.

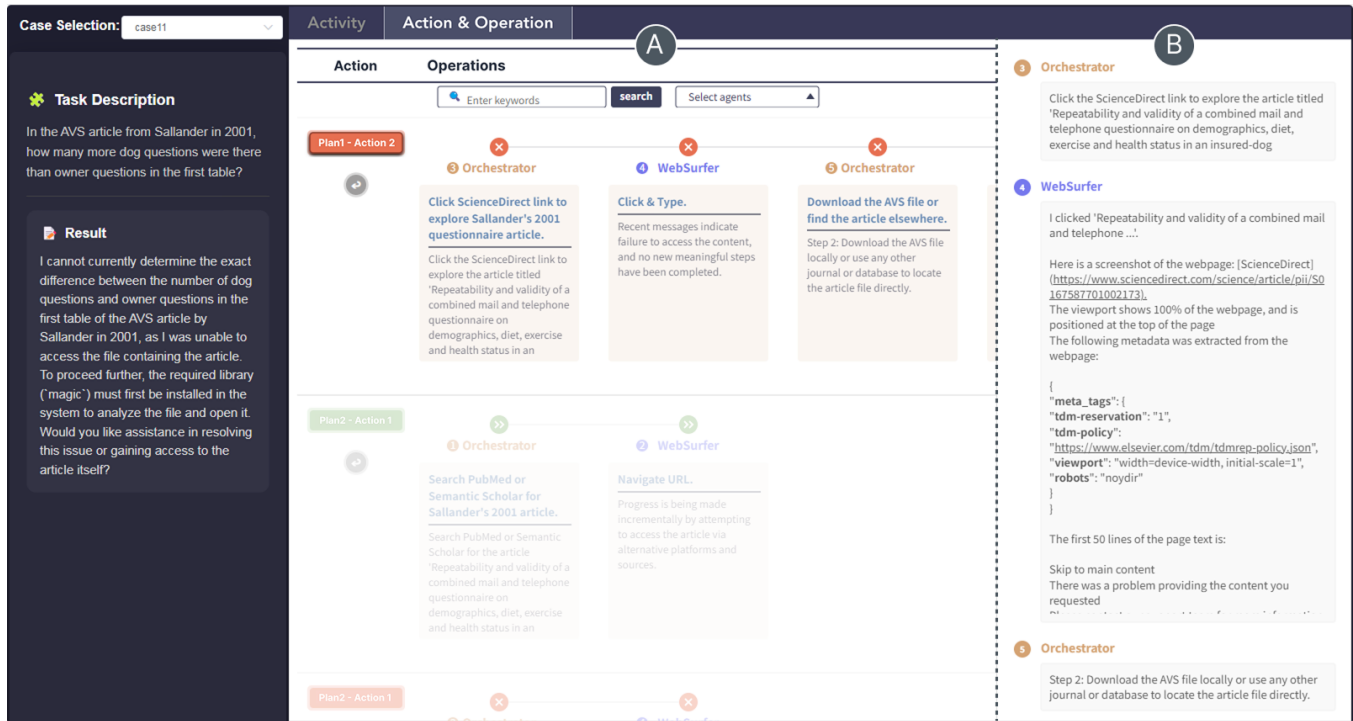


Figure 5: The Action View and the Operation View support developers in accessing detailed information across two layers. The left side of this view (A) displays action-level information, while the right side (B) showcases operation-level information.

5.1 Activity View

The Activity View (Fig. 4) provides AI developers with a high-level overview of activity execution, including overall progress and outcomes such as plans and their execution status. This view corresponds to the first layer of our multi-layered design (DC1), enabling developers to quickly assess whether the activity is proceeding as intended before drilling down into finer-grained details. This view can be regarded as the first level of agent behavior summary, including five types of information which have been introduced in Sec. 4.1 (i.e., plan failures and updating, actions in each plan, action status, agents for each action, and task progress).

Given the need to present multiple plans together with their associated action sequences, execution status, responsible agents, and progress updates, DiLLS adopts a table-based visualization (Fig. 4). This design choice directly operationalizes DC4 by structuring heterogeneous agent-generated information into clearly defined columns, enabling users to efficiently filter and navigate information while maintaining contextual relationships among plans, actions, and agents. Each type of information is displayed in a separate column, as shown in Fig. 4. The *Plan* column shows consecutive plans and their execution status. A warning icon ⚠ between two plans allows users to quickly identify where the previous plan got stuck. Hovering over the icon, users can view the multi-agent system's reasoning about the failure, such as possible causes and intended updates. By making these thoughts explicit at the point of failure, users can more easily understand the rationale behind plan

updates and examine whether the system's transition from failure diagnosis to recovery strategy is well justified (DC2).

When users seek more information about a particular plan, they can refer to the other columns to see how each plan is decomposed and executed. For example, Plan 1 is broken down into four distinct actions, each represented as a separate row in the table. Specifically, four types of information are provided (i.e., descriptions, execution status, involved agents, and progress), each displayed in a separate column. The *Actions* column highlights a concise summary of the actions in this plan, with the execution time of each action also presented. In addition, actions that are affected or updated in subsequent plans are prefixed with an update tag **updated**, allowing users to quickly recognize dependencies and revisions across plans. The *Status* column then encodes whether an action is completed **completed**, failed **failed**, or not started **not started**. In parallel, the *Agents* column lists the agents involved in each action.

Finally, to make agents' hidden assessments of task advancement interpretable to users, we explicitly externalize agents' internal progress judgments (DC2). Through this, users can quickly assess whether and why progress was or was not achieved, and whether the agents' own understanding of progress aligns with users' expectations of task advancement. Specifically, the *Progress* column depicts this type of information. Each marker represents the progress status of an operation within the action: making progress **»**, and no progress **×**. Considering that users often need to jointly reason about multiple agent behaviors rather than inspect them in isolation, we connect related operations (DC3). Specifically, DiLLS

segments progress into contiguous sequences based on progress signals, grouping consecutive operations that make progress and separating uninterrupted periods of no progress. These segments, with concise summaries, act as coherent sense-making units, enabling users to identify not only when progress occurred or stalled, but also what was concretely accomplished. Clicking one segment navigates to the corresponding details in the **Action & Operation View** (Sec. 5.2). This design aims to let developers trace agent behaviors more effectively and to selectively narrow their focus during the exploration process. Moreover, developers may need to jointly analyze agent behaviors that are not adjacent in the execution trace, such as operations performed by the same agent across different plans or operations pursuing the same intention across plan revisions. To support this need, *DiLLS* provides flexible filtering mechanisms that allow users to regroup and examine related behaviors beyond their temporal order (DC3).

5.2 Action View

The Action View (Fig. 5-A) corresponds to the second layer of our multi-layered design (DC1), enabling users to quickly comprehend how the system progresses step by step toward completing an action. For example, when reading an online PDF, developers may want to know which agents were involved and what sequence of steps they performed. At this stage, users are often concerned only with whether these operations succeeded or failed, rather than the full content of each step. In this way, the Action View serves as an effective overview of progress. If needed, they can then drill down into the Operation View to inspect details such as the exact text retrieved by a web agent.

First, users can click on the progress marker from the Activity View to reach this view. Since the action level displays the sequence of operations within a single action, we adopted a list format (Fig. 5 - the left side) (DC4), allowing users to easily follow the progression of operations and understand the internal workflow of each action. Each list corresponds to a segment from the activity level. We continue to use markers to show progress and the corresponding agents, which maintains visual consistency. Then, our system will display the operation description for each planning agent (i.e., the orchestrator) as it is responsible for issuing the execution commands. Specifically, our system highlights the summary of each operation instruction, followed by a detailed description of this action. This design choice aids in visual prioritization and draws attention to key details, allowing users to quickly identify important information. For the agent workers, we present their operation types in bold blue. Below that, we present the operation result summarization. This format supports quick scanning, enabling users to rapidly assess different operation types and their outcomes, improving the efficiency of information sensemaking.

Finally, we have also added several interaction features to help users explore more effectively. The system supports filtering interactions, similar to the Activity View. Users can search for corresponding operations using specific keywords or by selecting particular agents (DC3). Additionally, to support navigation, when users click a progress segment in the Activity View, the corresponding list in the Action View will be highlighted. At the same time, we increase the transparency of other unrelated lists. This allows users to focus

more on the content they care about. Next, users can also click each return icon  to go back to the Activity View, and we use an animation to indicate which row in the Activity View corresponds to the user's operation.

5.3 Operation View

Corresponding to the third layer of our multi-layered design (DC1), the Operation View provides users with the ability to review the log details generated by an agent's operation (Fig. 5-B). In this view, we parsed the log data to produce structured outputs, making it easier for users to review (DC4). Furthermore, when users click on a row in the Action View, the relevant log details will be displayed in the Operation View. In this way, users can easily navigate back and forth between different layers of information. Finally, to make it easier for users to view the URLs provided by the WebSurfer, we have directly set them as hyperlinks.

6 Evaluation

We conducted a within-subjects user study with 12 participants to evaluate the usability and effectiveness of our proposed system, *DiLLS*. According to the feedback from the formative study (Section 3), it is a common practice for developers to navigate the original log data using AutoGen Studio [6], CrewAI [5], or other tools to diagnose the behaviors of multi-agent systems. Therefore, in the control condition, participants were provided with observational log data. To mitigate the influence of unrelated variables, this baseline system simply replaced the Activity View and the Action & Operation View in *DiLLS* with a chronological presentation of log data, while retaining other components, such as the introduction and answers to the user query on the far left. In the experiment condition, we provided participants with our interactive system, *DiLLS*, to help them explore the agent behaviors.

Findings from the formative study suggested that diagnosing LLM-based multi-agent systems often involves substantial effort and uncertainty, prompting us to investigate how our system affects efficiency, confidence, and cognitive load. Accordingly, we propose the following three research questions: (RQ1) whether our system enables developers to identify and understand failures more efficiently; (RQ2) whether our system improves users' confidence in diagnosing multi-agent systems; and (RQ3) how users perceive the cognitive load of using our system, as well as their feedback on its functionality and design.

To operationalize these research questions, we evaluate diagnostic efficiency and effectiveness using task performance measures (e.g., the number and types of identified failures and inter-participant agreement), assess user confidence through post-task self-reported ratings, and examine cognitive load via NASA-TLX scores [13] and user experience via feature-level usability ratings. In addition, we conduct semi-structured interviews to collect qualitative insights that help contextualize these quantitative results.

6.1 Participants

We recruited 12 participants (P1-P12) through online advertisements on social media and word-of-mouth. The group included 6 females and 6 males, aged 24.17 ± 2.66 years. Participants came from

both academia and industry. They had diverse experiences in multi-agent system diagnosis: 12 had developed multi-agent systems for research purposes, and 3 had experience with projects in the industry. Two of them participated in one multi-agent project, five contributed to two projects, three worked on three projects, and two engaged in more than five related projects. Detailed demographic information of the participants is shown in Tab. 1.

Table 1: Participant demographics, including each participant’s ID, age, specialization, and the number of multi-agent development projects they had previously participated in.

ID	Gender	Age	Specialization	#Projects
P1	Male	24	Research	2
P2	Female	25	Industry & Research	3
P3	Male	24	Research	>5
P4	Female	22	Research	2
P5	Male	21	Research	1
P6	Female	23	Industry & Research	1
P7	Female	25	Research	3
P8	Female	21	Research	2
P9	Male	30	Research	>5
P10	Male	23	Research	2
P11	Male	24	Research	2
P12	Female	28	Industry & Research	3

6.2 Study Materials

We utilized the GAIA benchmark [39], a commonly used dataset to evaluate multi-agent system performance, to identify suitable cases for our experiment. For our multi-agent system, we used Magentic-One [9], one of the most widely used centralized multi-agent systems developed by Microsoft, which comprises five distinct agent roles. The Orchestrator manages the overall plans and executions, while the WebSurfer is responsible for tasks related to web searching. The FileSurfer performs operations associated with file handling, and the Coder engages in writing and debugging code. Finally, the Executor is tasked with executing the code. To obtain suitable cases for the study, we tested 50 questions from the GAIA dataset. Ultimately, we selected four error cases that have a balanced level of difficulty, considering the number of agents, actions, and operations involved. The first case involved the search question: “According to the US Bureau of Reclamation glossary, what does the acronym that shares its name with a book of the New Testament stand for” (denoted as Case 1). The second case asked: “In the AVS article from Sallander in 2001, how many more dog questions were there than owner questions in the first table” (denoted as Case 2). The third case asked: “In the NIH translation of the original 1913 Michaelis-Menten Paper, what is the velocity of a reaction to four decimal places using the final equation in the paper based on the information for Reaction 7 in the Excel file” (denoted as Case 3). The fourth case asked: “What is the average number of pre-2020 works on the open researcher and contributor identification pages of the people whose identification is in this file” (denoted as Case 4).

6.3 Procedure

We first introduced the construction of Magentic-One [9], allowing participants to understand important information about the composition of agents within the framework, which serves as the foundation for subsequent system diagnosis. Then, we invited each participant to complete two multi-agent system diagnosis tasks separately under the two conditions. Before the task starts in the system condition, we provide a 5-minute tutorial on *DiLLS* and give 15 minutes for participants to get familiar with it. We prepared four candidate cases with wrong outputs (see details in Section 6.2). Each participant completed two cases, one under the system condition and one under the baseline condition. To fully counterbalance which case was used in each condition, we generated all distinct pairs of two different cases (e.g., Case 1 used in the system condition and Case 2 in the baseline condition, or the reverse). The twelve resulting pairs were randomly assigned to the twelve participants. This ensured that every case appeared in both conditions across the study and that the case-condition assignments were fully counterbalanced. In each task, participants were given 20 minutes to explore the case and identify the error using either our system or the baseline system. We encouraged them to think aloud during these two sessions, and we recorded the video of each user study session with participants’ permission. At the end of each session, participants were asked to explain their identified errors and propose potential solutions. Participants then completed a 5-point Likert questionnaire assessing the effectiveness, usability, and overall user experience of the tool. Upon the completion of the two sessions, we further conducted a semi-structured interview to gather participants’ experiences and perceptions. Each participant received compensation of US\$20 for their time. Our project has also received IRB approval.

7 Results

During the user study, our participants were encouraged to think aloud as they worked through the tasks. After each task, we conducted semi-structured interviews. The full list of questions is included in Appendix B. Specifically, the think-aloud data were used to inform the adaptation and refinement of follow-up interview questions to probe these observations in more depth. The qualitative analysis focused on the interview data. All interview sessions were video-recorded and transcribed. Two authors independently coded the transcripts to generate initial codes, focusing on how the system’s features affected participants’ diagnostic efficiency, confidence, and cognitive load, each corresponding to one of the three research questions examined in Sec. 6. These evaluation dimensions were examined separately, as they were elicited through different sets of interview questions. Then, the two authors met to review and refine the codes, ultimately identifying representative codes and higher-level themes. Specifically, during the process, they found that some themes might span multiple evaluation dimensions. For instance, clearer tracing of the exploration process was described by participants as contributing to both higher confidence and lower perceived cognitive load. When themes overlapped, we assigned emphasis based on the dimension that participants most strongly foregrounded in their accounts. These themes serve as

qualitative findings that help contextualize the quantitative results, with participant quotations included as supporting evidence.

In this section, we first present the study results related to task performance. Next, we discuss user confidence in their diagnosis results, followed by an evaluation of their perceived load during the study. We then present additional important feedback provided by participants regarding our system features. Finally, we elaborate on the limitations of our user study. Specifically, we found that the collected quantitative metrics from users did not follow a normal distribution. Therefore, we used the Wilcoxon signed-rank test to calculate significance.

7.1 Task performance

DiLLS helps users find more failures in the multi-agent system within a fixed time budget. Specifically, under the baseline condition, each participant found an average of 2.25 (SD=1.23) failures. In contrast, under our system, participants identified an average of 3.67 (SD=1.84) failures. The results were statistically significant based on the Wilcoxon signed-rank test ($p < 0.05$), indicating that *DiLLS* is effective in assisting users in identifying failures, thus addressing **RQ1**. This is primarily because our system provides a hierarchical structure to organize agent behaviors. The different levels can help users identify various types of failures. Each structured layer highlights unsuccessful plans, actions, or operations, enabling our participants to quickly pinpoint agent behaviors that need detailed examination. Additionally, the transitions between layers allow users to easily recall the context.

To gain a deeper understanding of the identified failure points, we analyzed their types and examined the distribution of different types. First, three authors independently annotated the failures reported by our participants using an open coding approach. After this initial annotation, the authors convened to discuss their classifications. Ultimately, we established a final labeling schema. Each author then revised their previous annotations according to this schema. In cases where discrepancies arose, the authors engaged in discussions to determine the appropriate type for each failure. Here are the four types of failures identified in our study:

- **Problematic Planning:** This indicates that the generated plans, consisting of specific actions, were impractical for fulfilling the user request or the updates to these plans were inappropriate.
- **Action Skipping:** This refers to instances where the multi-agent system did not follow the predefined plan during its execution, potentially overlooking critical tasks.
- **Incorrect Operation Assignment:** This means that the LLM-based multi-agent system did not give proper and clear instructions for the next operation or wrongly assigned operations to the agents responsible for completing them.
- **Operation Completion Failure:** This indicates that the assigned operation was not successfully completed by the specific agent.

Fig. 6 shows the total number of failures identified across the four categories. *DiLLS* exhibits clear advantages in supporting users' detection of these failures, particularly *problematic planning* and *action skipping*, both of which require a higher-level understanding of the overall execution process.

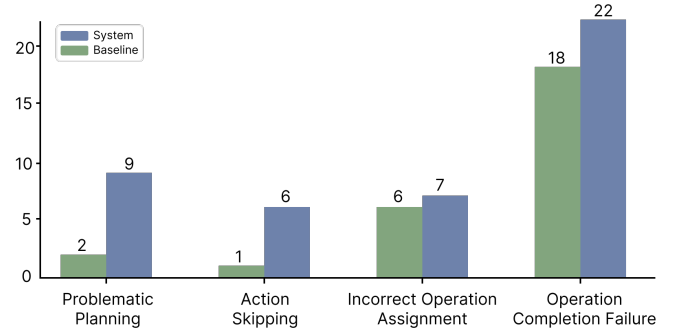


Figure 6: The distribution of different types of failures.

Additionally, we assessed inter-participant consistency in the identified failure types. Due to the limited sample size, conventional, inter-rater agreement metrics (e.g., Fleiss' Kappa) would not yield reliable estimates. Therefore, for the four examined cases, we instead report the proportion of failures identified by one, two, or all three participants (each case was independently reviewed by three participants under either the system or baseline condition). In the system condition, 17%, 25%, and 58% of failures were identified by one, two, and all three participants, respectively. In the baseline condition, the corresponding proportions were 50%, 30%, and 20%. These results indicate substantially higher agreement among participants when using the system compared with the baseline, suggesting stronger alignment in diagnostic judgments.

Finally, we summarize two reasons why our system helps participants more easily detect the first two failure types.

Our system strengthens information connections to support comparing actions across different plans. Our activity-level summary enables seven participants to focus more on whether failures occur in the iterations between plans and the correspondence between plan design and action execution. In the baseline, participants often became mired in operation details after understanding the plans, making it difficult to identify such issues in plan updates. For instance, P10 stated, “Plans and operations require constant back-and-forth scrolling for connection and comparison, so even when I want to check for action skipping or discrepancies, it is challenging and often inaccurate.” P4 also mentioned, “I believe identifying high-level failures related to overall execution logic is crucial, and I prefer to prioritize fixing these issues. If the execution logic is flawed, improving operational accuracy alone is unlikely to ensure task success. However, this can be quite challenging in practice. The two summary views effectively assist me in this process.” Additionally, our action-level summary serves a similar purpose. For instance, P4 noted that by summarizing all agent operations involved in completing an action, she can more easily identify differences and changes in the commands issued by the orchestrator.

Our system surfaces agents' internal reasoning to improve users' understanding of their behaviors. Five (of 12) participants mentioned that our system presents information they had not previously prompted the LLM-based agents to output, such as agents' perspectives on their own progress. This information helps them gain a deeper understanding of the reasons behind agents' executions, assisting in identifying potential failures. For example,

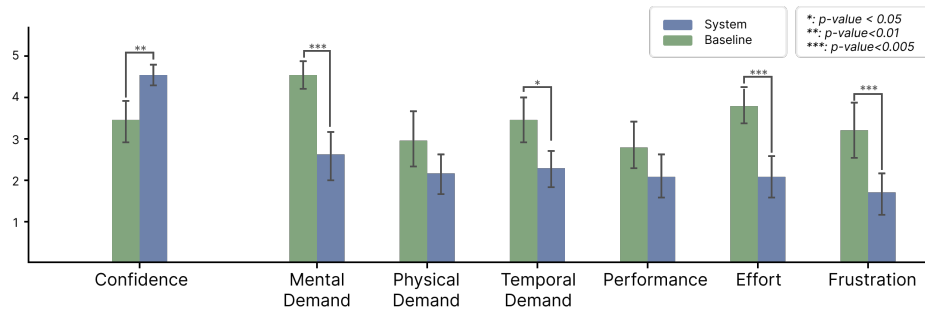


Figure 7: User confidence and perceived load.

during P6’s exploration, after the plan was updated and several operations executed, she did not see significant progress compared to the previous plan. “In this updated plan, I observed that the agents still found the URL for the AVS article but failed to retrieve specific information, which is no different from the previous plan’s execution.” However, the agentic system considered it as progress. Curious, she checked the details and discovered that when the agents evaluated whether they made progress, they did not seem to take all the information obtained from the previous plans into account. She stated, “This can be adjusted. We can enhance the agentic systems’ understanding of all previous actions to help them evaluate progress more comprehensively. This can be achieved by implementing a global progress tracking mechanism.”

7.2 User Confidence

We summarize participants’ self-reported confidence under the baseline and system conditions in Fig. 7. The results indicate that our participants reported higher confidence when using *DiLLS*, answering **RQ2**. Specifically, under the baseline condition, each participant reported an average confidence level of 3.42 ($SD = 1.00$). In contrast, under our system, participants identified an average confidence level of 4.50 ($SD = 0.52$). The results were statistically significant based on the Wilcoxon signed-rank test ($p < 0.05$), indicating that *DiLLS* is effective in enhancing user confidence in their identified failures and insights. Based on the user feedback, we summarize the main reasons for participants having higher confidence in their identified failures when using *DiLLS*.

Our system helps developers become aware of and trace their exploration process. The two types of behavior summaries we provide are an effective way to help developers continuously understand their current exploration progress. This allows users to have a clearer understanding of which parts they have explored and which they have not, reducing the likelihood of omissions. Ultimately, this leads to an increase in their confidence. We observed that six participants frequently recalled their exploration processes in a structured way. For instance, P3 initially identified the action he was investigating. Then he utilized the progress summary to quickly gauge the contextual information after he started to examine the operation under this action. He noted that the intuitive visual layout of each view aids him in tracing his exploration journey more effectively. P10 also stated, “*DiLLS* appears to guide me through this diagnosis process using three layers, making me feel that my diagnostic approach has become more systematic and comprehensive.”

Our system highlights the statuses of plans, actions, and operations to help users reconfirm and refine their initial hypotheses, if any. Six (of 12) participants highlighted the effectiveness of our system in emphasizing critical information, which plays a significant role in helping them confirm their initial hypotheses and even formulate more comprehensive hypotheses. For example, P11 noted, “When I use the baseline, I often worry that I might not have considered all the hypotheses which need validation, which leads me to constantly check.” P3 also mentioned that, as a developer with extensive experience, they might quickly skim through all the log data and form their own hypotheses. However, they are concerned that their experience could introduce certain biases. The system directly points out the erroneous actions or operations, making it easier for them to confirm and even refine their thoughts.

7.3 Perceived Load

We summarize participants’ perceived cognitive load through NASA Task Load Index (NASA-TLX) [13] under the baseline and system conditions in Fig. 7. We can observe that our system demonstrates advantages over the baseline across various dimensions, answering the first part of **RQ3**. We summarize two points to explain why our interactive system, *DiLLS*, is more advantageous than the baseline.

Our system provides a wealth of hints to guide users, allowing them to bypass redundant in-depth examination. Our activity-level summary outlines action statuses (i.e., completed, failed, and not started), effectively helping nine participants identify which actions to focus on. For instance, P10 mentioned, “In my previous work diagnosing multi-agent systems, the text volume was overwhelming, requiring me to scan everything to identify focus areas. This system highlights where the task is getting stuck.” Additionally, our progress summary also aids seven participants in quickly understanding the reasons for failures, which helps in formulating hypotheses about which operations to examine more closely. For instance, P8 stated, “The text summary for operations under an action is very useful. It helps me identify which of the several operations have issues, allowing me to focus on those specific operations.” At the same time, seven (of 12) participants also mentioned that our action-level summary highlights the success and failure of operations, which aids their failure location process. P2 said, “This information, combined with the summary of what each operation did, allows me to quickly understand the entire execution trajectory and identify where further investigation is needed.”

The seamless transition between layers allows users to easily recall context. Ten (of 12) participants appreciated the functionality that allows for seamless navigation and positioning between different levels. This ability to examine context at any time significantly reduces their perceived load during the diagnostic process. For instance, P7 noted that the transition animations are particularly smooth, and these animations help her understand where to focus her attention. P8 mentioned that the use of the same encoding method between different layers, such as the icons for success and failure, provides visual consistency. This consistency has aided in her transition between layers.

7.4 Further Analysis on User Experience Feedback

We analyzed user ratings of the usability for four features in our system: activity-level summary, action-level summary, operation-level details, and filtering function. The distribution of the user ratings can be seen in Fig. 8, addressing the second part of RQ3. We summarize two points explaining why our system achieves positive ratings through thematic analysis of the interview results. Finally, we also elaborate on their concerns regarding our system.

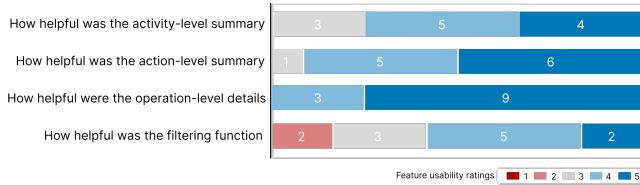


Figure 8: User perception towards the system.

Our system’s hierarchical organization of agent behaviors helps users notice failure types that they previously overlooked. Four (of 12) participants appreciated that our system helped them identify failure types they had not previously considered. P3 noted, “In the past, I never realized that task skipping could occur. However, this system has allowed me to efficiently detect when the agentic system starts the next action before the previous one has been executed.” P4 also mentioned, “These three layers provide a great taxonomy to help me identify and categorize failures. This systematic approach enables me to discover new types of failures.”

The filtering approach allows users to view successful and failed behaviors separately, enabling them to identify patterns and trends. Five (of 12) participants utilized the filtering feature to extract actions with different statuses. Their goal was to gain a comprehensive understanding of which actions frequently failed and which were successful, enhancing their understanding of multi-agent capabilities and facilitating hypothesis generation. This exploration allowed them to form hypotheses about the model’s performance and its underlying capabilities, which can help the diagnostic process. P1 articulated this sentiment well, stating, “This is similar to global explainability for AI models; I want to quickly understand the boundaries of a model’s capabilities—where it succeeds and where it fails.”

7.5 User Study Limitations

In this section, we mainly discuss some limitations of our user study. Our user study focuses solely on the initial step of debugging, helping users locate and understand failures by exploring agent behaviors (i.e., diagnosis). Therefore, after users generated hypotheses about how to improve multi-agent systems, we did not have them modify the system to test these hypotheses. This is primarily because actual debugging can be time-consuming, and we are unable to conduct this within the time constraints of our study. Finally, since diagnosis can be closely related to each individual’s experience, opinions, skills, etc., it is challenging to provide a ground truth to evaluate whether the participants’ results are correct. To alleviate this problem, we asked participants in the study to find corresponding evidence to support the failures they identified and to explain how they would like to address these failures.

8 Discussion

In this section, we summarize several design implications for interactive tools that can support multi-agent system diagnosis and understanding, based on findings from our user study. We also discuss the generalizability and limitations of our work, along with directions for future research.

8.1 Design Implications

Our study highlights two key design implications for multi-agent system diagnosis: balancing AI assistance with human judgment and leveraging contextual information for failure analysis.

Balancing Automation and User Agency in Multi-agent System Diagnosis. Our system automates the extraction, structuring, and summarization of planning and execution traces, tasks that are tedious for humans, while leaving failure interpretation and pattern summarization to users. This keeps human judgment central and reduces the risk of misleading hallucinations from fully automated diagnoses. At the same time, automation highlights potentially relevant behaviors across large volumes of data, easing cognitive load and helping users navigate complex traces. This balance allows AI to amplify user expertise without replacing it. Future work may explore more proactive AI suggestions, such as proposing possible root failures with explanations, but these features must be designed so users can validate or override AI outputs to maintain interpretability, trust, and a balanced human-AI workflow.

Failure Analysis with Context. During our user study, we discovered that some participants frequently relied on successful actions or similar failures to assist them in analyzing current failures. This contextual information serves as a powerful resource for diagnosis. For example, users might identify a successful configuration that resolved a similar issue in the past or draw parallels between the characteristics of a current failure and those of prior cases. Currently, our system facilitates context retrieval primarily by enabling users to easily access adjacent information or through basic filtering options. However, there is considerable room for improvement. Future designs should not only enhance search capabilities but also focus on how the interface is structured to support users in grouping contextual information effectively. Moreover, incorporating visual grouping tools or dashboards that display related actions and failures side by side could significantly aid users in their analysis.

By creating a more intuitive interface that emphasizes context, we can empower users to make informed decisions based on historical insights, ultimately improving their diagnostic outcomes.

8.2 Generalizability

In this paper, we introduce a framework for extracting, organizing, and presenting task-planning and execution information to support the diagnosis of centralized multi-agent systems. The framework is conceptually grounded in activity theory, leveraging its three-layer structure to frame and interpret agentic activities. Because this structure captures fundamental and inherent properties of activity organization, we anticipate that the conceptual framework may generalize to other multi-agent architectures. However, the technical means required to obtain the relevant information may differ substantially across architectures.

In principle, task execution histories can be used to explicitly extract or summarize activity organization, action planning, and operational sequences. However, the difficulty of obtaining such information varies across multi-agent architectures. In systems that follow a plan-then-execute paradigm, commonly found in centralized or hierarchical architectures, there is typically an orchestrator agent [29, 52], and its output history often provides direct access to planning and activity-organization information, enabling approaches similar to those used in this paper. In contrast, when planning and execution are interleaved, as in certain decentralized architectures or shared-message-pool systems [33, 59], obtaining high-level summaries becomes more challenging. In such cases, a bottom-up approach may be required, in which activity and action patterns are inferred from low-level traces, potentially supported by an observer agent that processes and aggregates the data.

8.3 Limitations and Future Work

We have discussed the limitations of our user study in Section 7.5. In addition, the use of LLMs for probing and summarizing text may also introduce inaccuracies. Although contemporary LLMs are generally robust for these tasks, errors can still occur. This risk can be mitigated, at least in part, through faithfulness evaluation methods [36]. In the rest of this section, we outline opportunities for future work in understanding and diagnosing multi-agent systems, using our system and extending beyond it.

Understanding Long-Term Benefits. Our user study is short-term and case-based. From a longer-term perspective, we aim to examine how the system influences practitioners’ diagnostic workflows. In particular, we seek to understand whether it can help practitioners develop more systematic and reusable diagnostic procedures. Moreover, we are interested in characterizing the patterns of model errors that emerge during extended diagnosis of a multi-agent system. Beyond insights tied to a single-case failure, we hope to uncover broader, systematic issues in system design, such as challenges in agent coordination or the lack of specific capabilities.

Broader Challenges in Understanding Multi-Agent Behaviors. Compared with single-agent systems, multi-agent systems introduce additional challenges for understanding their behaviors. In this paper, we focus primarily on their planning and execution behaviors as they work toward completing complex tasks. However, many

other important questions remain open—for example, how organizational behaviors emerge, how agents’ roles evolve during problem solving [56], and what patterns arise from conflicting objectives and negotiation behaviors [24]. Together, these challenges highlight the growing complexity of behavior analysis as multi-agent systems become more capable and autonomous. We encourage future research to deepen the investigation of these emerging dynamics and develop methods that better support practitioners in understanding and guiding multi-agent systems at scale.

Toward A Theory-based Approach to Investigating Multi-agent System Behaviors. Our use of Activity Theory (AT) focuses on its layered activity–action–operation structure to support practical diagnosis of multi-agent behaviors. However, AT offers far deeper analytical constructs, such as from aspects of mediation, community structures, rules, and contradictions that we do not explicitly model. These dimensions raise important questions about how agents coordinate, how tools and artifacts shape collective behavior, and how system-level tensions give rise to failures. While such inquiries extend beyond the scope of our diagnostic goals, situating our work within this broader AT landscape highlights promising directions for future research. We view *DiLLS* as an initial step toward more theory-informed analyses and hope it motivates deeper integration of AT concepts in the study of multi-agent system behaviors.

9 Conclusion

In this paper, we presented a framework and *DiLLS*, an interactive visualization system that facilitates the exploration of agent behaviors in LLM-based multi-agent systems for failure identification and understanding. Our system supports a layered exploration approach, grounded in Activity Theory, that enables comprehensive analysis of agent behaviors and allows AI developers to navigate activity-level, action-level, and operation-level information effectively. A user study with 12 AI developers demonstrates that our system can help practitioners identify and understand failures more efficiently and with greater confidence. Overall, our contributions provide a practical solution for diagnosing LLM-based multi-agent systems and pave the way for future research on developing more intuitive and effective tools for AI developers.

Declaration of the Use of AI

We used ChatGPT for grammar checking and to improve word expressions. The tool did not change the original meaning of the text, nor did it introduce any new references or knowledge.

Acknowledgments

We sincerely thank all our collaborators for their contributions. We also appreciate the support provided by the Hong Kong government. This research was partly supported by the General Research Fund of the Research Grants Council (RGC GRF) of Hong Kong (Grant #16210321), the Natural Science Foundation of Jiangsu Province (Grant #BK20241300), and the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery (Grant #RGPIN-2020-03966). In addition, we are extremely grateful to our reviewers for their careful reading of our manuscript and for providing detailed and constructive feedback.

References

- [1] Valtteri Ala-Salmi, Zeeshan Rasheed, Abdul Malik Sami, Zheyang Zhang, Kai-Kristian Kemell, Jussi Rasku, Shahbaz Siddeeq, Mika Saari, and Pekka Abrahamsson. 2025. Autonomous Legacy Web Application Upgrades Using a Multi-Agent System. arXiv:2501.19204 [cs.SE] doi:10.48550/arXiv.2501.19204
- [2] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.
- [3] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. In *Proceedings of the Twelfth International Conference on Learning Representations*, Vol. 2024. OpenReview.net, Online, 20094–20136. doi:10.48550/arXiv.2308.10848
- [4] Yongchao Chen, Jacob Arkin, Yang Zhang, Nicholas Roy, and Chuchu Fan. 2024. Scalable Multi-Robot Collaboration with Large Language Models: Centralized or Decentralized Systems?. In *Proceedings of the 2024 IEEE International Conference on Robotics and Automation*. IEEE, Yokohama, Japan, 4311–4317. doi:10.1109/ICRA57147.2024.10610676
- [5] CrewAI. 2024. CrewAI: The Leading Multi-Agent Platform. <https://www.crewai.com/>. Accessed: September 4, 2025.
- [6] Victor Dibia, Jingya Chen, Gagan Bansal, Suff Syed, Adam Fourney, Erkang Zhu, Chi Wang, and Saleema Amershi. 2024. Autogen Studio: A No-code Developer Tool for Building and Debugging Multi-agent Systems. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Miami, Florida, USA, 72–79. doi:10.18653/v1/2024.emnlp-demo.8
- [7] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang (Eric) Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, Article 156, 15 pages. doi:10.1145/3706598.3713581
- [8] Xianzhe Fan, Zihan Wu, Chun Yu, Fenggui Rao, Weinan Shi, and Teng Tu. 2024. ContextCam: Bridging Context Awareness with Creative Human-AI Image Co-Creation. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–17. doi:10.1145/3613904.3642129
- [9] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, et al. 2024. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. arXiv:2411.04468 [cs.AI] doi:10.48550/arXiv.2411.04468
- [10] Shanghua Gao, Ada Fang, Yepeng Huang, Valentina Giunchiglia, Ayush Noori, Jonathan Richard Schwarz, Yasha Ektefaie, Jovana Kondic, and Marinka Zitnik. 2024. Empowering Biomedical Discovery with AI Agents. *Cell* 187, 22 (2024), 6125–6151. doi:10.1016/j.cell.2024.09.022
- [11] Alireza Ghafarollahi and Markus J. Buehler. 2024. ProtAgents: Protein Discovery via Large Language Model Multi-agent Collaborations Combining Physics and Machine Learning. *Digital Discovery* 3, 7 (2024), 1389–1409. doi:10.1039/d4dd00013g
- [12] Alireza Ghafarollahi and Markus J. Buehler. 2024. SciAgents: Automating Scientific Discovery Through Bioinspired Multi-Agent Intelligent Graph Reasoning. *Advanced Materials* 37, 22 (2024), 2413523. doi:10.1002/adma.202413523
- [13] Sandra G Hart. 2006. NASA-task Load Index (NASA-TLX); 20 Years Later. In *Proceedings of the 2006 Human Factors and Ergonomics Society Annual Meeting*. Sage Publications, Los Angeles, CA, USA, 904–908. doi:10.1177/154193120605000909
- [14] Junda He, Christoph Treude, and David Lo. 2024. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2024), 1–30. doi:10.1145/3712003
- [15] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *Proceedings of the Twelfth International Conference on Learning Representations*. OpenReview.net, Online, 1–20. doi:10.48550/arXiv.2401.12954
- [16] Ziheng Huang, Sebastian Gutierrez, Hemanth Kamana, and Stephen MacNeil. 2023. Memory Sandbox: Transparent and Interactive Memory Management for Conversational Agents. In *Adjunct Proceedings of the 2023 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, New York, NY, USA, 1–3. doi:10.1145/3586182.3615796
- [17] Angel Hsing-Chi Hwang, John Oliver Siy, Renee Shelby, and Alison Lentz. 2024. In Whose Voice?: Examining AI Agent Representation of People in Social Interaction through Generative Speech. In *Proceedings of the 2024 ACM Designing Interactive Systems Conference*. ACM, New York, NY, USA, 224–245. doi:10.1145/3643834.3661555
- [18] Rolf Isermann. 2005. *Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance*. Springer, Berlin, Germany.
- [19] Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: Volume 1: Long Papers*. Association for Computational Linguistics, Toronto, Canada, 14165–14178. doi:10.18653/v1/2023.acl-long.792
- [20] Ellen Jiang, Kristen Olson, Edwin Toh, Alejandra Molina, Aaron Donsbach, Michael Terry, and Carrie J Cai. 2022. PromptMaker: Prompt-Based Prototyping with Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, Article 35, 8 pages. doi:10.1145/3491101.3503564
- [21] Hyoungho Jin, Seonghee Lee, Hyungyu Shin, and Juho Kim. 2024. Teach AI How to Code: Using Large Language Models as Teachable Agents for Programming Education. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–28. doi:10.1145/3613904.3642349
- [22] Minsuk Kahng, Ian Tenney, Mahima Pushkarna, Michael Xieyang Liu, James Wexler, Emily Reif, Krystal Kallarakal, Minsuk Chang, Michael Terry, and Lucas Dixon. 2024. LLM Comparator: Interactive Analysis of Side-by-Side Evaluation of Large Language Models. *IEEE Transactions on Visualization and Computer Graphics* 31, 1 (2024), 503–513. doi:10.1109/TVCG.2024.3456354
- [23] Tae Soo Kim, Yoonjoo Lee, Jamin Shin, Young-Ho Kim, and Juho Kim. 2024. EvalLM: Interactive Evaluation of Large Language Model Prompts on User-Defined Criteria. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, Article 306, 21 pages. doi:10.1145/3613904.3642216
- [24] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik S Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae W Park. 2024. MDAgents: An Adaptive Collaboration of LLMs for Medical Decision-Making. In *Proceedings of the Thirty-Seventh Conference on Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, Canada, 79410–79452. doi:10.5555/37377916.3740438
- [25] Kari Kuutti. 1996. *Activity Theory as a Potential Framework for Human-Computer Interaction Research*. MIT Press, Cambridge, MA, USA, 17. doi:10.7551/mitpress/2137.001.0001
- [26] Langchain-Ai. 2024. Langchain-ai/Langgraph-studio: Desktop App for Prototyping and Debugging LangGraph Applications Locally. <https://github.com/langchain-ai/langgraph-studio>
- [27] Aleksei N Leont'ev. 1974. The Problem of Activity in Psychology. *Soviet psychology* 13, 2 (1974), 4–33. doi:10.2753/RPO1061-040513024
- [28] A. N. Leont'ev and M J Hall. 1978. *Activity, Consciousness, and Personality*. Prentice-Hall, Englewood Cliffs, NJ, USA. 186 pages. <https://api.semanticscholar.org/CorpusID:141375009>
- [29] Bo Li, Junwei Ma, Kai Yin, Yiming Xiao, Chia-Wei Hsu, and Ali Mostafavi. 2025. Disaster Management in the Era of Agentic AI Systems: A Vision for Collective Human-Machine Intelligence for Augmented Resilience. arXiv:2510.16034 [cs.MA] doi:10.48550/arXiv.2510.16034
- [30] Fan Li, Su Han, Ching-Hung Lee, Shanshan Feng, Zhuoxuan Jiang, and Zhu Sun. 2025. A New Era in Human Factors Engineering: A Survey of the Applications and Prospects of Large Multimodal Models. *International Journal of Human-Computer Interaction* 41, 18 (2025), 1–14. doi:10.1080/10447318.2024.2446511
- [31] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative Agents for Mind Exploration of Large Language Model Society. In *Proceedings of the Thirty-Sixth Conference on Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., New Orleans, LA, USA, 51991–52008. doi:10.5555/3666122.3668386
- [32] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A Survey on LLM-based Multi-Agent Systems: Workflow, Infrastructure, and Challenges. *Vicinity Earth* 1, 1 (2024), 9. doi:10.1007/s44336-024-00009-2
- [33] Zhiyuan Li, Wenshuai Zhao, and Joni Pajarinen. 2025. Cooperative Multi-Agent Planning with Adaptive Skill Synthesis. arXiv:2502.10148 [cs.AI] doi:10.48550/arXiv.2502.10148
- [34] Yanna Lin, Shaojie Xu, Wenshuo Zhang, Yushi Sun, Zixin Chen, Yanjie Zhang, and Rui Sheng. 2025. A Survey of LLM-based Multi-agent Systems in Medicine. *Authorea Preprints* (2025). doi:10.36227/techrxiv.176089343.36199495/v1
- [35] Ziyi Liu, Zhengzhe Zhu, Lijun Zhu, Enze Jiang, Xiyun Hu, Kylie A Peppler, and Karthik Ramani. 2024. Classmeta: Designing Interactive Virtual Classmate to Promote VR Classroom Participation. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–17. doi:10.1145/3613904.3642947
- [36] Potsawee Manakul, Adian Liusie, and Mark Gales. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 9004–9017. doi:10.18653/v1/2023.emnlp-main.557
- [37] Sanwal Manish. 2024. An Autonomous Multi-agent LLM Framework for Agile Software Development. *International Journal of Trend in Scientific Research and Development* 8, 5 (2024), 892–898. doi:10.5281/zenodo.14179129
- [38] Vasilios Mavroudis and LangChain Contributors. 2024. LangChain v0.3. <https://python.langchain.com>. Accessed 2025.
- [39] Grégoire Mialon, Clémentine Fourier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAlIA: A Benchmark for General AI Assistants. In *Proceedings of the Twelfth International Conference on Learning Representations*. OpenReview.net,

- Online, 1–20. doi:10.48550/arXiv.2311.12983
- [40] Guangtao Nie, Rong Zhi, Xiaofan Yan, Yufan Du, Xiangyang Zhang, Jianwei Chen, Mi Zhou, Hongshen Chen, Tianhao Li, Ziguang Cheng, Sulong Xu, and Jinghe Hu. 2024. A Hybrid Multi-Agent Conversational Recommender System with LLM and Search Engine in E-commerce. In *Proceedings of the 18th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 745–747. doi:10.1145/3640457.3688061
- [41] Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. 2024. Skeleton-of-thought: Prompting LLMs for Efficient Parallel Generation. In *Proceedings of the Twelfth International Conference on Learning Representations*. OpenReview.net, Online, 1–20. doi:10.48550/arXiv.2307.15337
- [42] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 2023 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, New York, NY, USA, 1–22. doi:10.1145/3586183.3606763
- [43] Shuofei Qiao, Ningyu Zhang, Runnan Fang, Yujie Luo, Wangchunshu Zhou, Yuchen Eleanor Jiang, Chengfei Lv, and Huajun Chen. 2024. Autoact: Automatic Agent Learning from Scratch for QA via Self-planning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics: Volume 1: Long Papers*. Association for Computational Linguistics, Bangkok, Thailand, 3003–3021. doi:10.18653/v1/2024.acl-long.165
- [44] Zeeshan Rasheed, Malik Abdul Sami, Kai-Kristian Kemell, Muhammad Waseem, Mika Saari, Kari Systä, and Pekka Abrahamsson. 2024. Codepori: Large-scale System for Autonomous Software Development using Multi-agent Technology. arXiv:2402.01411 [cs.SE] doi:10.48550/arXiv.2402.01411
- [45] Omar Shaikh, Valentino Emil Chai, Michele Gelfand, Diyi Yang, and Michael S Bernstein. 2024. Rehearsal: Simulating Conflict to Teach Conflict Resolution. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–20. doi:10.1145/3613904.3642159
- [46] Shreya Shankar, JD Zamfirescu-Pereira, Björn Hartmann, Aditya Parameswaran, and Ian Arawjo. 2024. Who Validates the Validators? Aligning LLM-assisted Evaluation of LLM Outputs with Human Preferences. In *Proceedings of the 2024 37th Annual ACM Symposium on User Interface Software and Technology*. ACM, New York, NY, USA, 1–14. doi:10.1145/3654777.3676450
- [47] Leixian Shen, Haotian Li, Yun Wang, and Huamin Qu. 2024. From Data to Story: Towards Automatic Animated Data Video Creation with LLM-based Multi-agent Systems. In *Proceedings of the 2024 IEEE VIS Workshop on Data Storytelling in an Era of Generative AI*. IEEE, St. Pete Beach, FL, USA, 20–27. doi:10.1109/GEN4DS63889.2024.00008
- [48] Mirac Suzgun and Adam Tauman Kalai. 2024. Meta-prompting: Enhancing Language Models with Task-agnostic Scaffolding. arXiv:2401.12954 [cs.CL] doi:10.48550/arXiv.2401.12954
- [49] Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-agent Collaboration: Harnessing the Power of Intelligent LLM Agents. arXiv:2306.03314 [cs.AI] doi:10.48550/arXiv.2306.03314
- [50] Ian Tenney, Ryan Mullins, Bin Du, Shree Pandya, Minsuk Kahng, and Lucas Dixon. 2024. Interactive Prompt Debugging with Sequence Saliency. arXiv:2404.07498 [cs.CL] doi:10.48550/arXiv.2404.07498
- [51] Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, and Hoang D Nguyen. 2025. Multi-Agent Collaboration Mechanisms: A Survey of LLMs. arXiv:2501.06322 [cs.AI] doi:10.48550/arXiv.2501.06322
- [52] Harold Triedman, Rishi Dev Jha, and Vitaly Shmatikov. 2025. Multi-Agent Systems Execute Arbitrary Malicious Code. In *Second Conference on Language Modeling*. OpenReview.net, Online, 1–15. doi:10.48550/arXiv.2503.12188
- [53] Voiceflow. 2025. Voiceflow: Build AI Agents. <https://www.voiceflow.com/>. Accessed: September 4, 2025.
- [54] Hongyu Wan, Jinda Zhang, Abdulaziz Arif Suria, Bingsheng Yao, Dakuo Wang, Yvonne Coady, and Mirjana Prpa. 2024. Building LLM-based AI Agents in Social Virtual Reality. In *Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, Article 65, 7 pages. doi:10.1145/3613905.3651026
- [55] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science* 18, 6 (2024), 186345. doi:10.1007/s11704-024-40231-1
- [56] Qian Wang, Tianyu Wang, Zhenheng Tang, Qinbin Li, Nuo Chen, Jingsheng Liang, and Bingsheng He. 2025. MegaAgent: A large-scale autonomous LLM-based multi-agent system without predefined SOPs. In *Findings of the 2025 Conference of the Association for Computational Linguistics*. Association for Computational Linguistics, Vienna, Austria, 4998–5036. doi:10.18653/v1/2025.findings-acl.259
- [57] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. Autogen: Enabling Next-gen LLM Applications via Multi-agent Conversation. arXiv:2308.08155 [cs.AI] doi:10.48550/arXiv.2308.08155
- [58] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. Os-copilot: Towards Generalist Computer Agents with Self-improvement. arXiv:2402.07456 [cs.AI] doi:10.48550/arXiv.2402.07456
- [59] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. 2025. Agentnet: Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems. arXiv:2504.00587 [cs.MA] doi:10.48550/arXiv.2504.00587
- [60] Yu Zhang, Jingwei Sun, Li Feng, Cen Yao, Mingming Fan, Liuxin Zhang, Qianying Wang, Xin Geng, and Yong Rui. 2024. See Widely, Think Wisely: Toward Designing a Generative Multi-Agent System to Burst Filter Bubbles. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–24. doi:10.1145/3613904.3642545

A Formative Study Interview Questions

Here, we list the pre-defined questions for the formative semi-structured interview. During the interview, if participants' initial responses did not fully cover the intended aspects of a question, we would also provide follow-up prompts accordingly.

(1) Latest Experience with Multi-Agent System Development

- Please recall and describe your latest experience of building a multi-agent system.

(2) The Process of Multi-agent System Diagnosis

- Did you encounter any failures in your multi-agent systems? If so, how did you diagnose the causes of the failures?

(3) The Challenges of Multi-agent System Diagnosis

- Which parts of the diagnostic process did you find difficult, time-consuming, or cognitively demanding?
- What difficulties did you face during the diagnostic process?
- Did you use any tools to support your diagnostic process? If so, what tools or interfaces did you use? How did these tools or interfaces benefit your diagnostic process?
- Do you have any expectations and concerns for the existing tools or interfaces for multi-agent system diagnosis?

(4) Others

- Is there anything else about diagnosing multi-agent systems that we have not covered?

B User Study Interview Questions

We listed the main interview questions used in the user study. Participants were also encouraged to tell their stories and experiences outside these structured questions.

(1) User Experience During the Process

- Please respectively describe your experiences completing the multi-agent system diagnosis task using the baseline system and using *DiLLS*.
- Compared with the baseline system, what do you consider the main strengths and weaknesses of our proposed system?
- How did the baseline system and *DiLLS* each affect your confidence in the identified failures?
- How did using the baseline system and *DiLLS* influence your perceived load while completing the diagnosis task?

(2) User Perception Towards *DiLLS*

- How did you perceive the usability of *DiLLS*? How did you perceive the usability of the four main features of *DiLLS*, including the activity-level summary, action-level summary, operation-level details, and filtering function?
- How did you perceive the usefulness of *DiLLS*? How did you perceive the usefulness of the four main features of *DiLLS*, including the activity-level summary, action-level summary, operation-level details, and filtering function?

(3) Future Use and Limitations

- What limitations did you observe in our system that could be addressed in the future?
- Is there anything else you would like to add?